ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ГОРОДА МОСКВЫ
«Колледж связи № 54» имени П.М. Вострухина

ОТЧЕТ О ВЫПОЛНЕНИИ

по дисциплине

“Внедрение и поддержка компьютерных систем”

Практической работы №3

«Подключение Flask и первый маршрут (Hello World)»

Выполнил: ст.гр. 3-ИСП11-43

ФИО: Юхимчук Евгений Викторович

Специальность: 09.02.07

Информационные системы и программирование

Проверил: преподаватель Кандыбаева

Москва

2026

Практическая работа №3. Подключение Flask и первый маршрут (Hello World)

Ссылка на репозиторий: https://github.com/gangdexyuhi/client-server-practice
Папка работы: practice_03_client_server/

Цель работы

Познакомиться с микрофреймворком Flask, запустить локальный веб-сервер, создать маршруты и разобраться, как браузер (клиент) взаимодействует с сервером по протоколу HTTP.

Используемые технологии

- Python 3.13
- Flask 3.1 (веб-фреймворк, встроенный сервер разработки Werkzeug)
- Протокол HTTP, методы GET и POST, коды ответов 200/404/405
- Браузер для проверки страниц

Структура проекта

```
src/
├── init_db.py, check_db.py, crud_console.py   # из работ №1-2
├── app.py              # Flask-приложение
├── requirements.txt    # зависимости (flask)
└── templates/          # папка для HTML-шаблонов (пока пустая)
```

Описание реализации

В файле app.py создаётся объект приложения Flask(__name__). Декоратор @app.route связывает URL с функцией-обработчиком: маршрут / обслуживает функция index(), которая возвращает HTML-страницу со ссылкой, а маршрут /about — функция about(), которая возвращает текст о проекте. Сервер запускается командой python app.py в режиме отладки (debug=True) на порту 5000.

```
from flask import Flask

# __name__ помогает Flask найти папку templates рядом с этим файлом
app = Flask(__name__)


@app.route('/')
def index():
    """Главная страница."""
    return """
    <html>
        <head><title>Список задач</title></head>
        <body>
            <h1>Мой список задач</h1>
            <p>Скоро здесь будет список задач из базы данных.</p>
            <p><a href="/about">О проекте</a></p>
        </body>
    </html>
    """


@app.route('/about')
def about():
    return "Это учебный список задач на Flask. Практическая работа №3."


if __name__ == '__main__':
    # debug=True - автоперезагрузка при изменении кода и подробные ошибки
    app.run(debug=True, port=5000)
```

Flask установлен командой pip install flask. Список зависимостей записан в requirements.txt, чтобы проект можно было развернуть командой pip install -r requirements.txt.

Ход работы и результаты запуска

1. Запустил сервер командой python app.py. В терминале видно, что включён режим отладки и сервер слушает адрес http://127.0.0.1:5000. Ниже в журнале — запросы браузера и автоматические перезапуски сервера после изменения app.py.

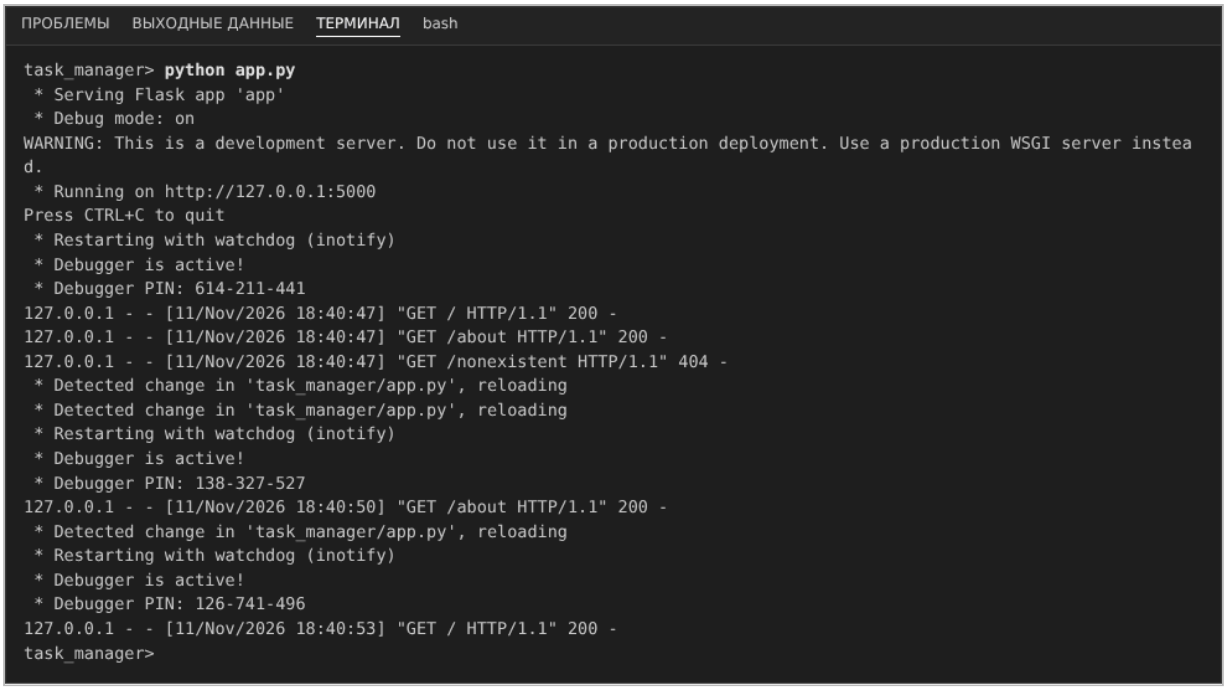


Рисунок 1 — Запуск сервера Flask и журнал запросов

2. Открыл главную страницу http://127.0.0.1:5000/ — отображается заголовок «Мой список задач» и ссылка «О проекте».

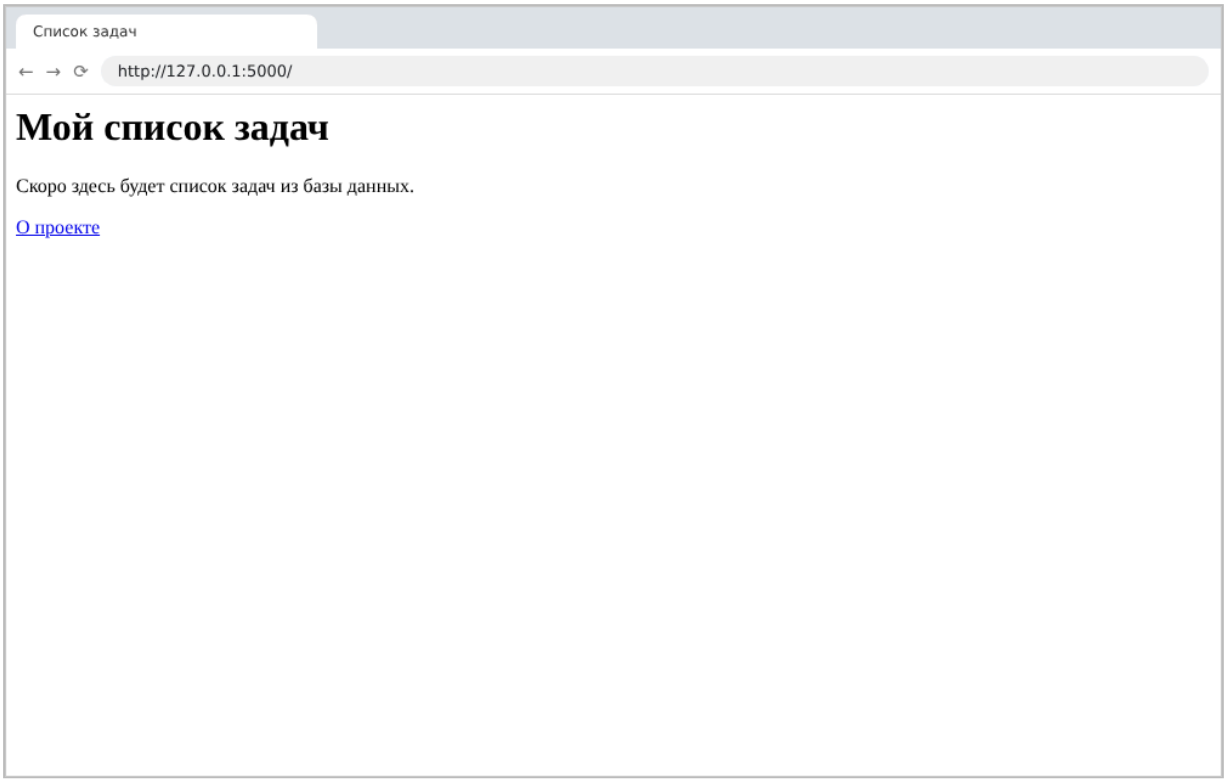


Рисунок 2 — Главная страница

3. Перешёл по ссылке «О проекте» — открылась страница /about.

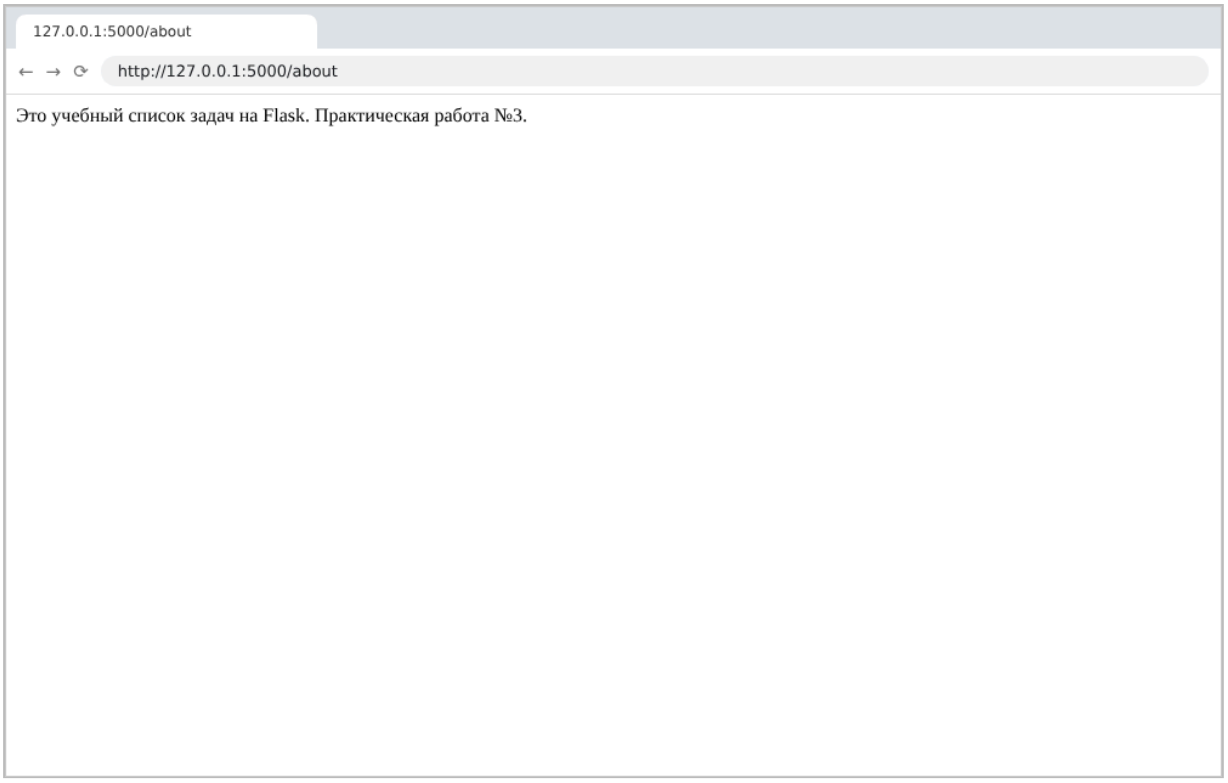


Рисунок 3 — Страница /about

4. Открыл несуществующий адрес /nonexistent — сервер вернул стандартную страницу 404 Not Found.

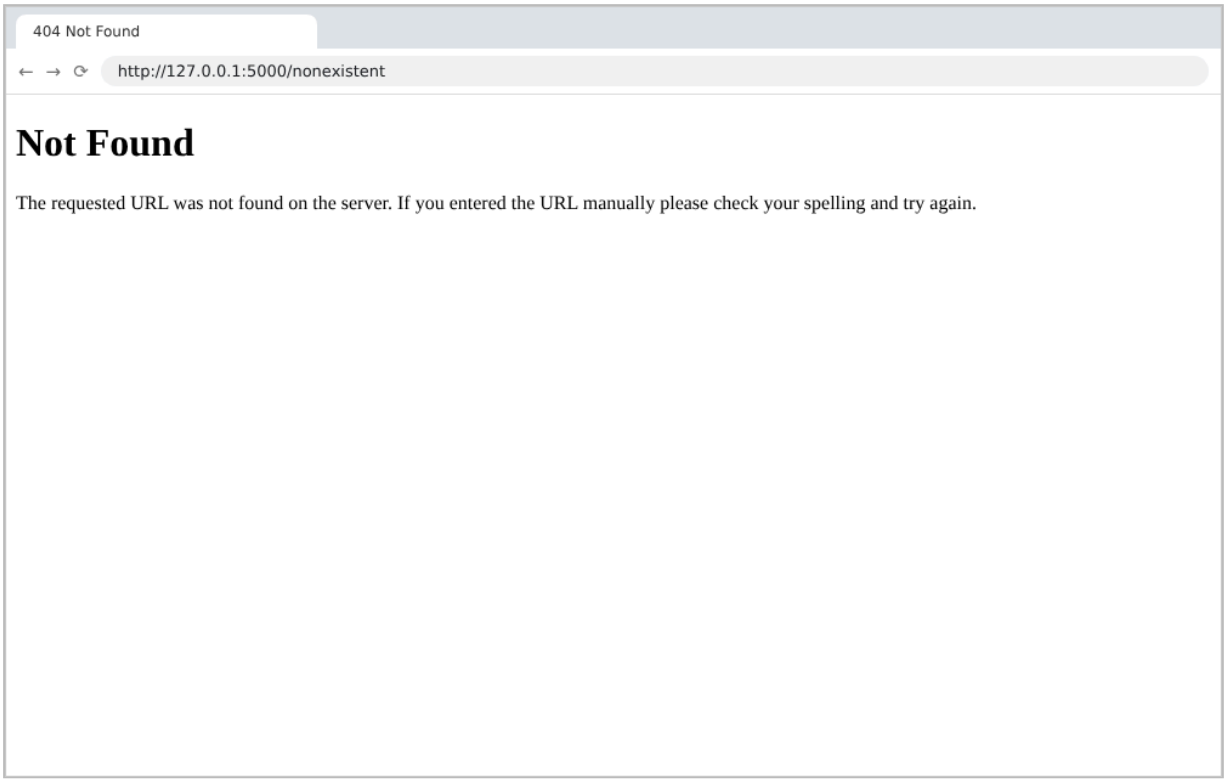


Рисунок 4 — Ответ 404 на несуществующий адрес

5. Проверил автоперезагрузку: не останавливая сервер, изменил текст в функции about() и сохранил файл. В журнале появилась строка Detected change in app.py, reloading, а после обновления страницы отобразился новый текст.

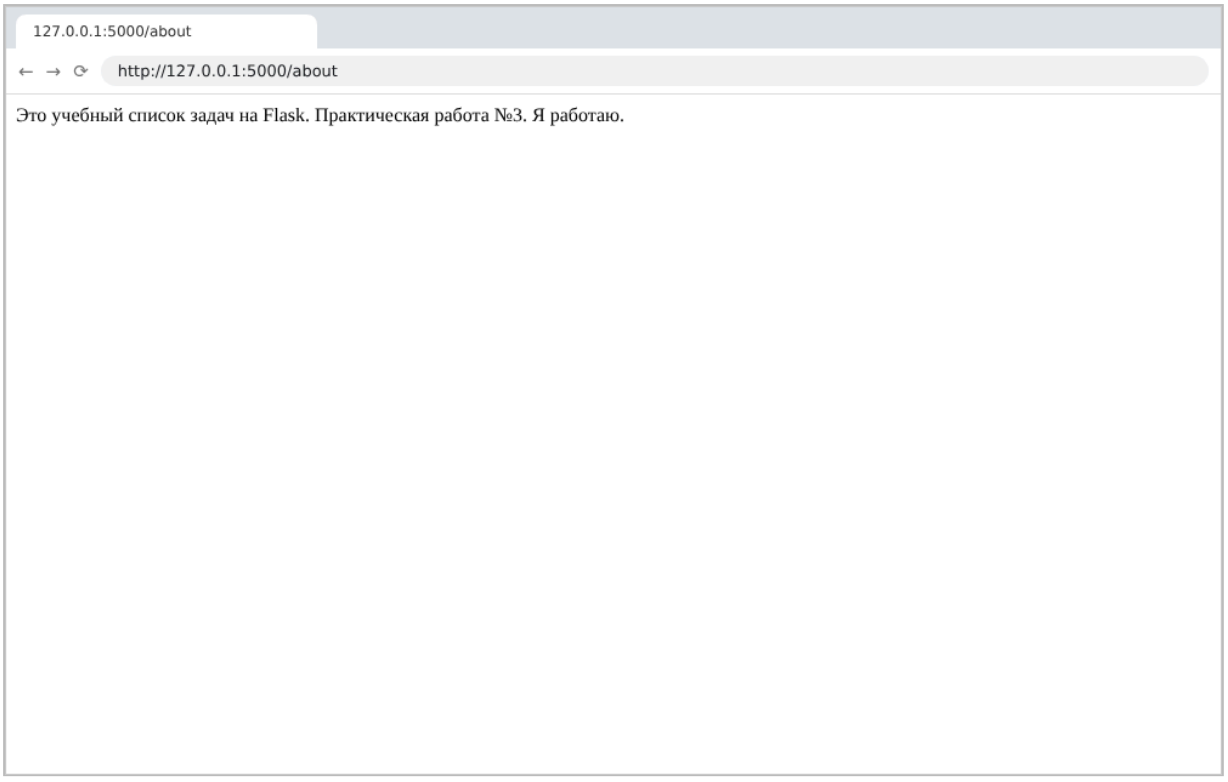


Рисунок 5 — Страница /about после изменения кода без перезапуска сервера

Примеры запросов и ответов

Таблица 1 — Примеры HTTP-запросов и ответов сервера

Запрос	**Код ответа**	**Заголовок Location**	**Результат**
GET /	200 OK	—	HTML главной страницы
GET /about	200 OK	—	Текст о проекте
GET /nonexistent	404 NOT FOUND	—	Маршрут не найден
POST /	405 METHOD NOT ALLOWED	—	Маршрут принимает только GET

Ответы на контрольные вопросы

1. Что делает декоратор @app.route('/')?

Регистрирует функцию как обработчик URL /. Когда на сервер приходит запрос к этому адресу, Flask вызывает функцию и отправляет

браузеру то, что она вернула.

2. Чем GET отличается от POST? Какой метод используется по умолчанию?

GET используется для получения данных (открытие страницы), параметры передаются в URL. POST используется для отправки данных на сервер (форма), данные передаются в теле запроса. По умолчанию маршрут Flask принимает только GET (и HEAD); на POST к такому маршруту сервер ответил 405 Method Not Allowed.

3. Зачем нужен debug=True и почему его нельзя использовать на реальном сервере?

Режим отладки автоматически перезапускает сервер при изменении кода и показывает подробную трассировку ошибок прямо в браузере. На реальном сервере это опасно: трассировка раскрывает код и пути, а встроенный отладчик позволяет выполнять произвольный Python-код на сервере.

4. Что означает адрес 127.0.0.1:5000?

127.0.0.1 — адрес localhost, то есть собственного компьютера, 5000 — номер порта, на котором Flask принимает запросы. Сервер доступен только с этого компьютера.

5. Почему при открытии /nonexistent мы получаем 404, а не ошибку Python?

Flask сначала ищет подходящий маршрут. Если ни один маршрут не совпал с URL, функция-обработчик вообще не вызывается, а фреймворк сам формирует стандартный HTTP-ответ 404 Not Found. Python-кода с ошибкой не выполняется.

6. Почему папка для HTML-шаблонов должна называться именно templates?

Это соглашение Flask: render_template по умолчанию ищет файлы в папке templates рядом с модулем приложения (путь определяется через __name__). При другом имени возникнет ошибка TemplateNotFound, если не указать папку явно параметром template_folder.

7. Что произойдёт, если запустить два сервера Flask одновременно на одном порту?

Второй сервер не сможет занять уже используемый порт и завершится с ошибкой Address already in use. Нужно остановить первый сервер (Ctrl+C) или запустить второй на другом порту.

Вывод

В ходе работы я установил Flask, создал приложение с двумя маршрутами и запустил локальный сервер. Я проверил ответы сервера на разные запросы (200, 404, 405) и убедился, что в режиме отладки сервер сам перезапускается после изменения кода. Я понял схему клиент-серверного взаимодействия: браузер отправляет HTTP-запрос, Flask находит маршрут, вызывает функцию и возвращает ответ.